Final Report Machine Learning

Master's in Biomedical Engineering

João Afonso - 95938, Luís Farrolas - 95942

October 31, 2023

1 Part 1 - Regression with Synthetic Data

1.1 First Problem - Linear Regression

1.1.1 Introduction and analysis of the data

In the first problem, a test and training set were provided, with the outcome only available for the training set. The first problem had a small sample size, with only 15 samples for 10 different features, which led to problems such as overfitting. To overcome this problem, the creation of a validation set through a split of the training set was discussed and ultimately discarded, as the group believed the training dataset was too small for said course of action, which may have introduced unwanted variability in the end results by removing valuable information from training the model.

Before creating a linear model to fit the dataset, a heatmap displaying the correlation values between variables was created. The heatmap (as seen in Figure 1) demonstrated that some variables (namely "var0", "var7" and "var9") were moderately negatively correlated [2] with other variables such as "var0" and "var3", moderately highly positively correlated [2] thus warranting regularization further down in the model creation process.

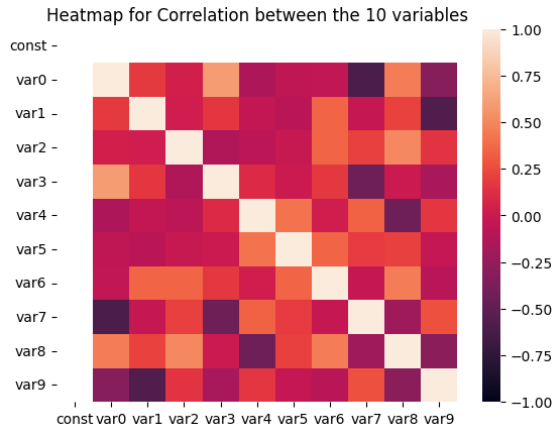


Figure 1: Heatmap regarding Task 1's dataset correlation between variables

The regression models were subject to k-fold cross-validation. Leave-one-out cross-validation was used in all the models, as smaller values for k could have created unwanted variability. Techniques such as regularization were also implemented to tackle the previously mentioned problems.

1.1.2 Ordinary Least Squares

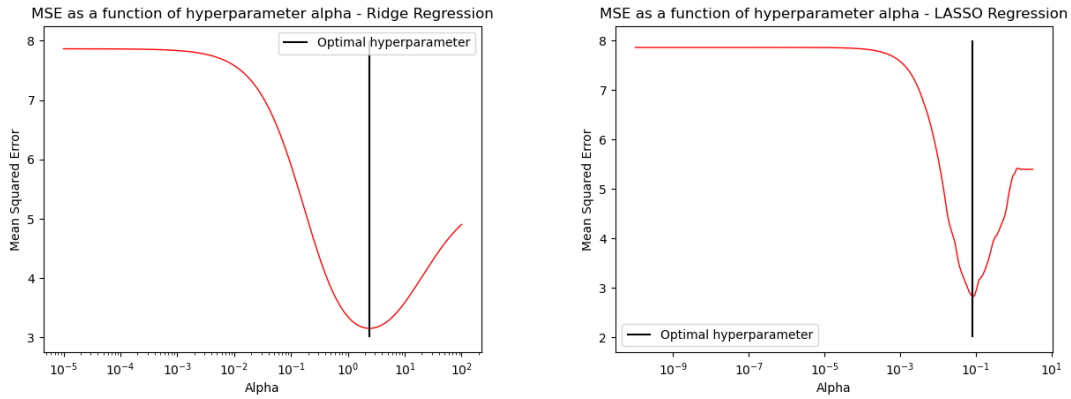
An Ordinary Least Squares linear regression model was first created and trained by using the *LinearRegression()* function present in scikit-learn [3] with the provided training sets, yielding the following β coefficients: [0.36, -0.26, 0.99, -0.01, 0.21, 1.64, -1.35, -0.13, 0.72, 0.77].

1.1.3 Ridge and LASSO Regression

Through correlation analysis, it was discovered that some variables in the dataset were moderately correlated (coefficients of $\pm$ 0.5-0.7) [2]. In order to tackle this problem, thus creating a more accurate model, regularization was implemented, with two methods being explored: LASSO (or l1) and Ridge (or l2) regularization.

Ridge regression was implemented by resorting to *RidgeCV()* present in scikit-learn and presented a steady behaviour in terms of MSE as a function of the alpha parameter, as seen in Figure 2a. The β coefficients determined by Ridge regression for variables 'var0' through 'var9' go as follows: [-0.01 , -0.54, 0.73, -0.27, 0.20, 1.21, -0.59, 0.14, 0.38, 0.55]. The higher the beta coefficient, the more significant the predictor. Hence, with a certain level of model tuning, we can determine the best variables that influence this problem.

LASSO regularization, performed with *LassoCV()* present in scikit-learn [3], was then tested, performing feature selection by determining its optimal hyperparameter (Figure 2b) and attributing the following β coefficients to variables 0 through 9, respectively: [0, -0.38, 0.98, -0.22, 0, 1.58, -0.75, 0, 0.04, 0.58], eliminating influence from features 'var0', 'var4' and 'var7'.



(a) Ridge Regression **(b)** LASSO Regression

Figure 2: Variation of the Mean Squared error, with the increase of the hyperparameter alpha. The objective was to identify the optimal hyperparameter alpha for this problem.

1.1.4 Results and Discussion

As seen in Table 1, the values for MSE indicate a better performance when regularization is applied, with Ridge regularization performing better than LASSO; the variable correlation analysis allied to a small number of variables would indicate that the LASSO model's sparsity could lead to the loss of valuable information as it would have deleted the influence of several variables, thus performing worse on its own. Although the validation measure for the task was SSE, both SSE and MSE were used, for both regression tasks, to check for the model's performance since the last one provides a direct measure of how far the errors are from the actual values and is robust to possible changes in dataset size (eg. train-test splits).

Table 1: Method performance - Task 1

Method	Optimal hyperparameter	R2	MSE
OLS	-	**0.93**	7.86
Ridge	2.41	0.89	**3.35**
LASSO	0.08	0.91	5.31

As seen in Figures 2a and 2b, Ridge regression behaves more regularly than LASSO when near their optimal parameters, consequently being more robust than LASSO to changes to the input dataset, which, when allied to the MSE results present in table 1 should warrant its use in the creation of the model. At the time of submission, however, we selected LASSO regression as the chosen model, as its R^2 was better and we believed that this metric better illustrated the model's accuracy.

Test set score: SSE = 1816.37. Ranking: 22/136. Overall, we believe that the model was well-trained and solved the proposed problem satisfactorily. After a more in-depth analysis, however, we do believe that we could have achieved better results by performing feature selection through Lasso regularization followed by a Ridge regression to the shrunken dataset.

1.2 Second Problem - Linear Regression with Two Models

1.2.1 Introduction and analysis of the data

Simple linear regression aims to find the best-fit line that describes the linear relationship between some input variables X and the target variable Y. This has some limitations as in real-world problems, there is a high probability that the dataset may have outliers, resulting in biased model fitting. To overcome this limitation of the biased fitted model, robust regression was introduced, a type of regression that deals with such datasets by being designed to be less sensitive to outliers.

In this second problem, a dataset containing data generated by two distinct linear models was provided, and the task was to create two models able to estimate the target values for both instances.

1.2.2 Ordinary Least Squares

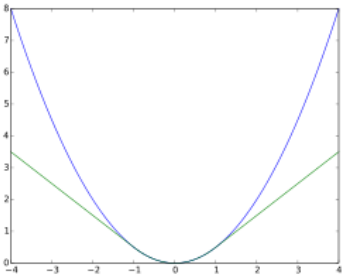
In order to establish a baseline, an Ordinary Least Squares regression was applied to the dataset, with no separation being performed, yielding the following β coefficients: [0.51, 0.49, 0.15, 0.25].

1.2.3 Dataset Separation: Clustering vs. Robust Regression (RANSAC and Huber)

Our first approach to this quasi-classification problem involved the creation of two clusters using functions *make_blobs()* and *KMeans()* present in scikit-learn [3], yielding two clusters, as requested, seen in Figure 4a. Regression models were then created by fitting the resulting subsets and each model's final MSE was evaluated, as seen in section 1.2.4.

RANSAC regressionn, implemented with *RANSACRegressor()*, present in scikit-learn [3], is an algorithm that selects a random number of samples and trains them as inliers, then tests all other samples on the trained model, selecting the inliers based on the mean absolute deviation (MAD, the metric chosen for our model). The model keeps getting trained with the new inliers until a limit of iterations (in our case, 13) is reached. From the list of supposed inliers and outliers, two masks were created to separate the data into two different subsets, ideally similar in size. The resulting subsets can be seen in Figure 4b. Regression models were then created by fitting the resulting subsets and each model's final MSE was evaluated, as seen in section 1.2.4.

Huber's regression, implemented with *HuberRegressor()*, present in scikit-learn [3], uses a similar loss function to the least squares penalty for small residuals, but on large residuals, so its penalty is lower and increases linearly rather than quadratically. We implemented Huber as a means to assess RANSAC's performance since both are Robust Regression methods. Regression models were then created by fitting the resulting subsets and each model's final MSE was evaluated, as seen in section 1.2.4.

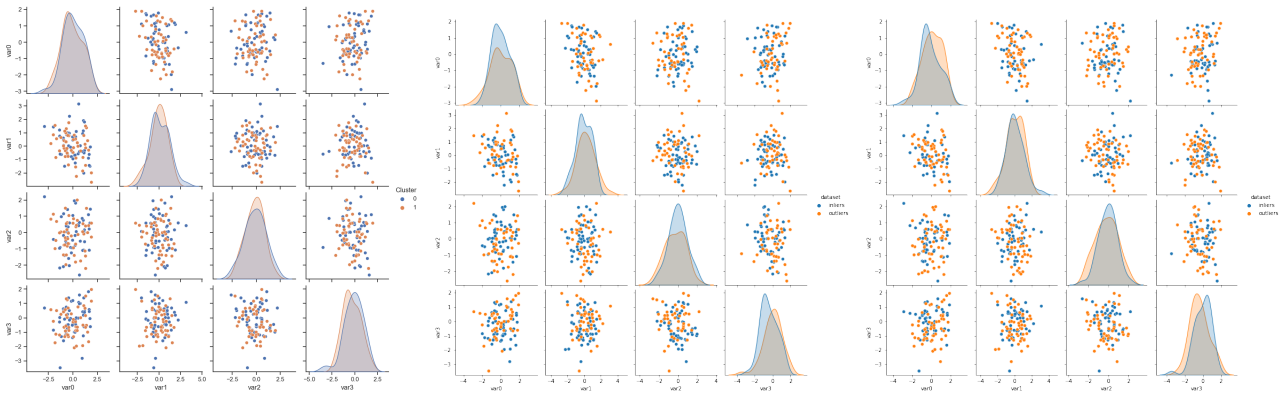


$$L_\delta(a) = \begin{cases} \frac{1}{2}a^2 & \text{for } |a| \le \delta, \\ \delta \cdot \left(|a| - \frac{1}{2}\delta\right), & \text{otherwise.} \end{cases}$$

(a) Huber loss function(green), with $\delta = 1$, compared to the regular squared error loss(blue)

(b) Huber Loss function. δ controls the number of samples that should be classified as outliers, while α affects the strength of the squared L2 regularization.

Figure 3: Characteristics of the Huber Loss function.



(a) KMeans clustering **(b)** RANSAC regressor **(c)** Huber regressor

Figure 4: Separation of the data for the methods tested: Clustering and Robust regressors; dots in blue and orange represent the inliers and outliers of the models, respectively.

1.2.4 Results and Discussion

Due to RANSAC relying on initial randomness, final results relying on RANSAC will always present a source of variability, exacerbated for small enough datasets. In order to control this randomness, many random states were tested, aiming to achieve balanced datasets in terms of size while minimizing the R2 values for the final regressions. Due to our inital tests' unsatisfactory results, we tried to scale and normalize the data before separation, resorting to the *RobustScaler()* function, present in scikit-learn [3], chosen for its ability to standardize the data while being robust to outliers. This may have proven to be a mistake, as in order to avoid any more interference with the data, the group set all subsequent *fit_intercept* parameters in each of the regression functions to "False", yielding poor results. The results obtained below, Table 2, as well as the graphs in Figures 4a, 4b and 4c have had all of the *fit_intercept* parameters corrected to "True".

These results suggest that the best methods to use were RANSAC + Ridge, as Huber's separation led to some problems in the fitting of the second model, not being able to construct two satisfactory subsets of similar size from our dataset; this could be due to the fact that RANSAC is better at dealing with outliers in the y direction [3]. The clustering method, although yielding results comparable to Huber in regards to total MSE, was much worse at creating valid subsets, as Huber's performance woes can be attributed to problems in regularization of the second subset - when not regularized, its performance was comparable to RANSAC. K-Means, on the other hand, is not appropriate for non-linearly separable data as it creates clusters based on each data point's

Euclidean distance from one another, thus the clustering technique would have had to explore other clustering methods to attain better results.

Table 2: Method performance in terms of MSE - Task 2

Method	Model 1 MSE	Model 2 MSE	Total MSE
OLS	-	-	139.91
Clustering + OLS	71.89	80.20	152.09
Clustering + Ridge	49.19	56.77	105.96
Clustering + LASSO	75.06	78.16	153.22
RANSAC + OLS	1.65	0.90	2.55
RANSAC + Ridge	1.45	**0.67**	**2.12**
RANSAC + LASSO	5.93	3.85	9.78
Huber + OLS	1.77	0.90	2.67
Huber + Ridge	**1.15**	106.16	107.31
Huber + LASSO	6.33	135.36	141.69

Test set score: SSE = 391.2387. Ranking: 92/135. Overall our model while not being completely ineffective at separating the two subsets, left a lot to be desired in terms of accuracy. Its shortcomings could be mostly attributed to having set all *fit_intercept* parameters to "False" in fear of interfering too much with the dataset's distribution of information, which led to poorer fit from each of the used regressors. The use of other unsupervised learning algorithms could also be a possible alternative for the separation of the subsets, which contained non-linearly separable information, thus warranting the exploration of appropriate methods.

2 Part 2 - Image Analysis

2.1 First Task

2.1.1 Introduction and analysis of the data

In this task, a binary classification problem was proposed. A dataset containing 6254 observations, of which 5358 were 0 (nevus) and 896 were 1 (melanoma) was presented.

Compared to the previous exercise, image data analysis is a more complex and time-consuming problem. The RGB images present 3 channels and have dimensions of 28*28, which can translate into 2352 pixel values per image. Therefore, not only using Regression models can be very time-consuming, but images have complex relationships between pixels, which are not suitable for linear models. In this section of the work, models such as Naive Bayes, Logistic Regression, KNN, Cart, and SVM will be discussed as well as convolution neural networks (CNN's), a specialized deep learning approach for the problem. Before training any model, the data was standardized, changing the pixel values to range from 0 to 1, to reduce computational work.

2.1.2 Naive Bayes approach

Naive Bayes, implemented with function *GaussianNB()*, present in scikit-learn [3], is a basic but effective probabilistic classification model in machine learning that draws influence from Bayes Theorem. Since the dataset is unbalanced, two techniques were used to balance the data:

1. SMOTE algorithm, an oversampling method, that generates synthetic data for the minority class by creating new instances similar to the existing ones. The k_neighbour parameter sets up the surroundings for the creation of the new samples(we used 1 neighbour since more would create a smoothing effect on the new images).

2. RandomUndersampling, a technique that reduces the samples of the overrepresented class, can be helpful in a very large dataset, which was not the case.

The results regarding the balanced accuracy were slightly better with SMOTE oversampling when compared to undersampling and no data preprocessing (71.49% vs 70.74% vs 71.01%; tables 3, 4 and 5, respectively), results that could have been expected given the imbalance of the dataset, removing an important amount of the data with the undersampling technique, would yield in poor training of the naive Bayes algorithm. Note that this preprocessing technique was used in all the following models except for the CNN. Note that no further improvements were made to this model since it is not optimal for image classification.

Table 3: SMOTE confusion matrix for training = 20%.

ACTUAL \ PREDICTION	Neg	Pos
Neg	**747**	327
Pos	47	**130**

Table 4: Undersampling confusion matrix for training = 20%.

ACTUAL \ PREDICTION	Neg	Pos
Neg	**767**	307
Pos	53	**124**

Table 5: Unbalanced confusion matrix for training = 20%.

ACTUAL \ PREDICTION	Pos	Neg
Pos	**744**	330
Neg	48	**129**

2.1.3 Logistic Regression approach

The *LogisticRegression()* function from scikit-learn [3] is used mainly for binary classification and uses the sigmoid function to return the probability of a label. It also uses a cross-entry cost function and gradient descent. However, due to its simplicity, this method also did not produce good results for a complex image classification problem as the one presented, with a balanced accuracy of 71.96% with SMOTE oversampling.

2.1.4 K-Nearest Neighbors approach

Using the KNN approach, the parameter k_neighbours of the *KNeighborsClassifier()* function, present in scikit-learn [3], was set to 7 after some tries with other neighbour values. The result were poor since this algorithm is not fine-tuned for image classification, since the prediction gives the output as the class with the majority in the 'k' nearest neighbours, ignoring the specific features of the image. The balanced accuracy was 72.79%.

2.1.5 Classification And Regression Trees approach

The CART algorithm is a decision tree where each fork is split into a predictor variable and each node at the end has a prediction for the target variable. This model can create over-complex trees if the user is not aware with pruning, and also create high variance, which lead to poor models for image classifications, when compared to simple classification problems, where it can perform well. Using the *DecisionTreeRegressor()* function, present in scikit-learn [3], the balanced accuracy of this algorithm was 67.14%

2.1.6 Support Vector Regression

SVM was the best-performing algorithm by far when compared to the previous ones mentioned, with a balanced accuracy of 79.24%. Unlike other Regression models that try to minimize the error between the real and predicted value, the *SVR()* function, present in scikit-learn [3], tries to fit the best line within a threshold value, while also being very effective even with high dimensional data, such as the images presented in this task.

2.1.7 Convolutional Neural Network approach

Before entering the model, pre-processing of the data was performed, only on the training subset. The two techniques mentioned before, SMOTE and RandomUndersampling, were performed, as well as a new, third one involving data augmentation. The best performance was obtained through the combination of SMOTE oversampling and data augmentation. This allowed not only for the increase of the undersampled class, with similar images (k_neighbours=1), but also to diversify the inputs for the model with rotations, translations,

changes in contrast, and zoom; for all of these inputs, tests were performed to achieve the best-performing model.

A Convolutional Neural Network or CNN is an approach used especially for image recognition and classification, being used in Deep Learning algorithms. It is an algorithm divided into three main layers, to create complex features with the end goal of identifying the object in question. The layers implemented in the project, depicted in Figure 5, are as follows:

1. Convolutional layer(in red): In the model presented, our group tried both 2 and 3 layers, yielding better results with 3. A convolutional layer uses convolutions to create a feature map over the entire image, indicating the locations and strength of a detected feature in an image. These layers possess activation functions that will determine the output of the layer. Two different activation functions were tested during the process, 'tanh', which is a hyperbolic tangent activation function. The other activation function tested was the default in every deep learning network, the 'relu', which has the problem of being half rectified. The fact that our dataset has images with different approximations, and background colors for the same type of label, we believe that justifies the 3-layer preference, also with the tanh activation function.
2. Pooling layer(in green): MaxPooling layer was added, to downsample the input along its spatial dimensions. Other Pooling type layers were tested, such as the AveragePooling and the MinPooling with worse results, since in this case, the MaxPooling function allows the detection of features such as edges and corners. A Dropout layer was implemented in the end, with several values and distributions through the model being tested.
3. Fully connected layer(in yellow): The last part of the model, where the classification occurs and the input neurons connect to the output neurons. It has two sublayers, the .Flatten(), which takes the output of the previous layers, "flattens" them, and turns them into a single vector that can be an input for the next stage, and the .Dense(), which takes the inputs from the feature analysis and applies weights to predict the correct label, outputting the final probability for each label. The last layer activation function considered was 'sigmoid' since it was designed to predict for binary classification (output is between 0 and 1).

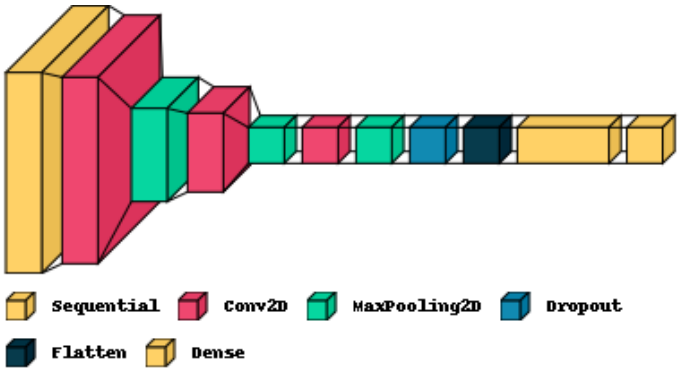


Figure 5: Model with all the layers considered for the binary classification problem - Task 3.

When training this model, three optimizers were tested. SGD, which subtracts the gradient multiplied by the learning rate from the weights. Adam optimizer is also an SGD method that is based on adaptive estimation of first-order and second-order moments. AdamW optimizer has an added method to decay weights, "Decoupled Weight Decay Regularization", which as the name appeals, adjusts the weight decay term to appear in the gradient update. Literature says that a fine-tuned Adam optimizer should always perform better than an SGD optimizer[5]. Therefore, after a few tests, and considering both performance and time, there was a consensus for the AdamW optimizer, with a learning_rate=0.001 and a weight_decay=0.0004. Also, EarlyStopping was implemented, with the objective of reducing overfitting. This allowed us to train the model with many epochs

and stop it as soon as it started to overfit, i.e., the validation loss would stop decreasing for a certain number of epochs. For this model, the best performing model was run for 100 epochs, using early stopping with patience of 20, considering binary cross-entropy as our loss function, depicted in Figure 6b.

2.1.8 Discussion of the results

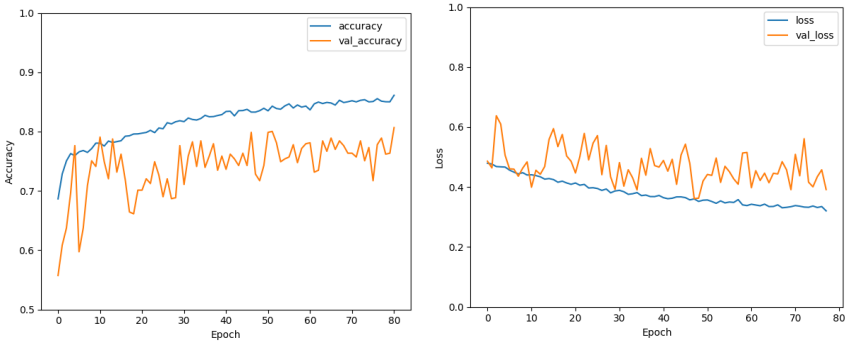
Support Vector Regression (SVR) and Convolutional Neural Networks (CNNs) performed significantly better than the other models for this image classification task, as seen in Table 6. SVR's effectiveness should be attributed to SVMs' ability to handle high-dimensional data, while our CNN, carefully designed and tuned for image analysis, demonstrated CNNs' known ability to handle such complex tasks as image analysis. The other methods ended up falling short of these results: Logistic Regression's simplicity could explain its shortcomings while CART may have performed worse due to the high variance inherent to tree creation. Naïve Bayes may have underperformed when in comparison to CNN and SVM due to its (naïve) assumption that no correlation exists between data points which is not the case for pixels in an image, whose values are highly correlated.

Table 6: Method performance (Balanced Accuracy) - Task 3

Method	Balanced Accuracy (%)
Naïve Bayes	71.52
Logistic Regression	71.96
K-Nearest Neighbors	72.79
CART	67.14
SVM	79.24
CNN	**81.74**

From these results, it should come as no surprise that our CNN was the model chosen, even though there is a case to be made for SVM, which performed comparably well while being less resource-intensive.

Test set score: Balanced Accuracy (In_Domain) = 0.8052; Balanced Accuracy (Out_Domain) = 0.5750. Ranking: 27/128. In regards to in-domain dataset classification, we believe that our model performed well. A classification considering an out-of-domain dataset was also performed, which showed poor results for the model we trained regarding balanced accuracy. This could be related to the fact that we performed a tuning for the hyperparameters specific to this model, turning the model into a non-generalized one. Our model could have probably performed even better if we used a data scaler such as *standardscaler()*, present in scikit-learn [3], which could yield better all-around performance for other datasets, by centering and scaling the training data. Possible future improvements may also lie in the exploration of pre-trained architectures such as *ResNet* or *VGG* as starting points for our final model.



(a) Accuracy (Ratio of correct predictions for total number of predictions)

(b) Loss Function (Binary Cross-Entropy)

Figure 6: Accuracy and Loss Function values for every training epoch of our CNN.

2.2 Second Task

2.2.1 Introduction and analysis of the data

In the second part of the task, there was the introduction of a multi-label problem (6 labels), and also the introduction of 2 sources of images, from 2 different hospitals. Therefore, we are presented with a multiclass-classification problem, where for the simple solution, a few changes from the previous CNN must be made. Firstly, in the output layer of the defined model, instead of a 1-neuron output, the final Dense layer will receive 6 neurons, with softmax as the activation function. This activation function is used in multinomial logistic regression since it scales numbers into probabilities, returning the most probable class.[4] The same preprocessing method utilized in section 2.1.7 was used to balance the classes, that had the initial distribution (5362, 890, 116, 2305, 990, 966), for classes 0 through 5, respectively, i.e. SMOTE technique with data augmentation. Finally, the loss function was also changed to categorical cross-entropy.

With these changes, the same model with the same layers as the previous exercise was built, and classification was obtained with a balanced accuracy of 74 % for the validation set. Given the additional information that classes 0, 1, and 2 belong to a different dataset from the others, there was the possibility of creating a first model to predict the class of the images to add another feature to the final model. Therefore, a simple binary classification model was built to predict the dataset of origin from the test images, with a high accuracy (99,9%). With that information, two paths were taken:

- In the first approach, two models were built, one for classifying classes 0 to 2 and the other for classes 3 to 5. The images were first classified in the binary classifier, and from there, two separate datasets were classified into two separate models. Similar layers, with less number of neurons were used since the dataset was divided, however different augmentations were used, which slightly boosted the performance of the model since the images had different contrasts and magnifications. This approach showed improved results, especially for the second trained model, where its balanced accuracy was 91%. The first model, however, only provided 79% of accuracy, which combined yielded a model with a total balanced accuracy of 85%, with total results seen in Figure 7.
- The second approach used the concept of multi-input CNN, where instead of the CNN input being just the images, there was also a second input for the dataset classification, which was introduced after flattening the features from the images, depicted in figure 8. This approach allowed the model to have one more feature to train, showing improvement compared to the first model, proving however not as effective as the one mentioned before, with a balanced accuracy of 82%.

In this last model, we chose to have a higher number of neurons for the first layer since the model was dealing with the whole data when compared to the split model, which proved to be effective yielding better results. Also, other approaches on these layers were tested, such as having small dropout layers after each convolutional layer [1], which did not show improvement. Batch normalization [6] was tested since it helps normalize the output of the hidden layers to address internal covariate shifts, showing however worse results.

2.2.2 Discussion of the results

Test set score: Balanced Accuracy = 0.83763656 . Ranking: 69/135 We believe both our models were well implemented, as the results high balanced accuracies, confirmed by 7, the confunsion matrix for the first approach, which yielded the most accurate model out of the two. Possible improvements may involve the use of pre-trained models, or finding a better architecture, especially for the first dataset, where maybe fewer layers would improve the training.

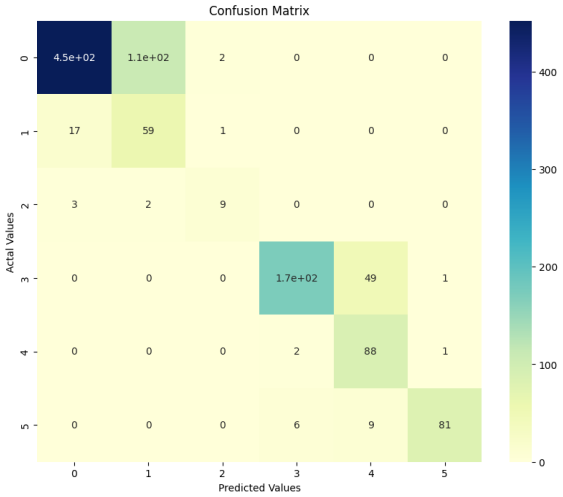


Figure 7: Confusion Matrix regarding the results obtained for the first approach, with the combination of the two sub-models.

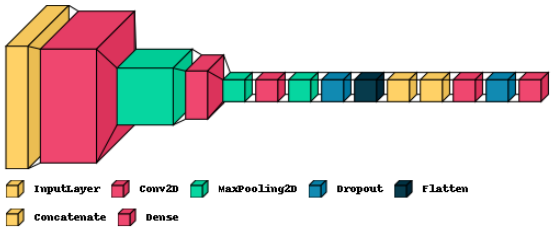


Figure 8: Model with multi-inputs regarding the images and their determined classes.

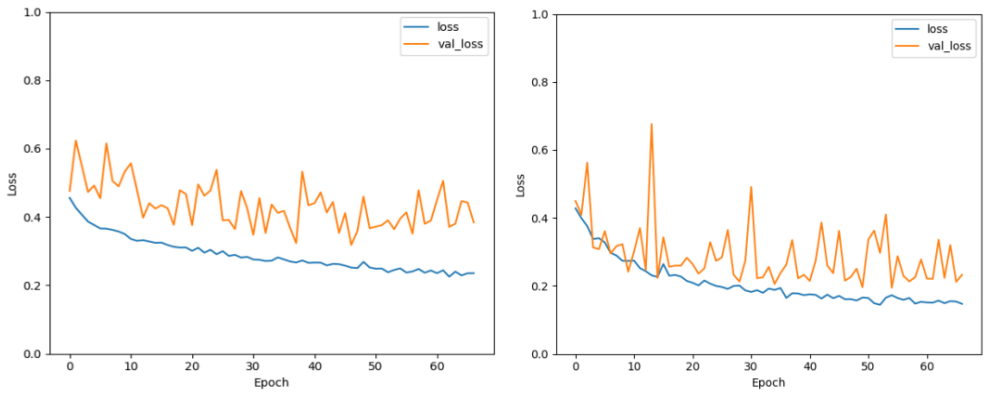


Figure 9: Loss Functions (Categorical crossentropy) values for the first approach's two final models. On the left: Model for classes 0-2. On the right: Model for classes 3-5.

References

[1] Jason Brownlee. *Dropout Regularization in Deep Learning Models with Keras - MachineLearningMastery.com*. 2022. URL: https://machinelearningmastery.com/dropout-regularization-deep-learning-models-keras/.

[2] Keith Calkins. *Applied Statistics - Lesson 5 Correlation Coefficients*. 2005. URL: https://www.andrews.edu/~calkins/math/edrm611/edrm05.htm.

[3] scikit-learn developers. *User Guide*. URL: https://scikit-learn.org/stable/user_guide.html.

[4] Kiprono Elijah Koech. *Softmax Activation Function — How It Actually Works*. 2020. URL: https://towardsdatascience.com/softmax-activation-function-how-it-actually-works-d292d335bd78.

[5] Sieun Park. *A 2021 Guide to improving CNNs-Optimizers: Adam vs SGD*. 2021. URL: https://medium.com/geekculture/a-2021-guide-to-improving-cnns-optimizers-adam-vs-sgd-495848ac6008.

[6] Keras Team. *Keras documentation: BatchNormalization layer*. 2015. URL: https://keras.io/api/layers/normalization_layers/batch_normalization/.